

Manual Unificado de Configuración

Proyecto Contrabajo

Versión 5 — Mayo 2026

Onboarding completo: desarrollo nativo + despliegue con Docker + frontend Android

Índice

1. Introducción	3
1.1 Cómo está organizado este manual	3
1.2 Glosario rápido	3
1.3 Comparación rápida: cuándo usar cada ruta	4
1.4 Cambios desde la versión anterior (v4)	4
2. Pre-requisitos comunes a ambas rutas	4
2.1 Software	4
2.2 Hardware sugerido	5
2.3 Cuentas y accesos	5
PARTE II — Ruta Nativa (desarrollo)	6
3. Clonar el proyecto	6
4. Instalar y configurar VS Code	6
4.1 Instalación	6
4.2 Extensiones obligatorias	6
4.3 Abrir el proyecto en VS Code	7
5. Crear las bases de datos	8
5.1 Ubicación de los scripts	8
5.2 Pre-requisitos	12
5.3 Crear el login admincontrabajo	12
5.4 Orden de ejecución	12
5.5 Otorgar permisos al login	13
5.6 Ejecución con sqlcmd (PowerShell)	13
5.7 Verificación	14
6. Configurar variables de entorno (.env + launch.json)	14
6.1 Archivo .env	14
6.2 .vscode/launch.json	22
7. Arrancar los servicios nativos	33
7.1 Spring Boot Dashboard (VS Code, recomendado)	33
7.2 Run and Debug (F5)	33
7.3 Contrabajo Launcher.ps1 (Windows)	33
7.4 Maven desde terminal	33
8. Exponer puertos con Dev Tunnels	33
8.1 Crear un túnel	33
8.2 Exponer los 4 puertos a la vez	33
8.3 Usar la URL en el frontend Android	34
PARTE III — Ruta Docker (correr el stack)	35
9. Configurar memoria WSL para Docker	35
9.1 Crear o actualizar .wslconfig	35

9.2 Verificar	35
9.3 Aplicar	35
10. Clonar el repo orquestador y configurar credenciales	35
10.1 Clonar el repo orquestador	35
10.2 Crear el .env	36
10.3 Obtener Personal Access Token de GitHub	37
10.4 Completar el .env	37
11. Levantar el stack con Docker	38
11.1 Primer arranque	38
11.2 Veces siguientes	40
11.3 Actualizar a una versión nueva	40
11.4 Levantar desde Docker Desktop (sin terminal)	40
11.5 Orden de arranque automático	41
12. Verificar y operar el stack Docker	41
12.1 Desde Docker Desktop	41
12.2 Endpoints y Swagger	41
12.3 Verificar las 3 BDs	41
12.4 Detener el stack	41
12.5 Persistencia de datos	42
PARTE IV — Frontend Android	43
13. Configurar y compilar la app Android	43
13.1 Abrir el proyecto	43
13.2 Configurar URLs según escenario	43
13.3 Compilar y ejecutar	44
PARTE V — Operación, troubleshooting y apéndices	45
14. Conectar SSMS a SQL Server (Docker)	45
15. Verificación end-to-end	45
16. Flujo de desarrollo con Docker	46
16.1 Cuándo recompilar/repullar	46
16.2 Acceder a una consola dentro de un contenedor	46
16.3 Debug remoto JVM desde VS Code (sobre contenedores)	46
16.4 Logs filtrados	46
17. Errores comunes y soluciones	47
17.1 Memoria / WSL / Docker Desktop	47
17.2 Variables de entorno y .env	47
17.3 BDs y SQL Server	47
17.4 Conexión Android / red	48
17.5 Otros	49
18. Referencia rápida de comandos	49
18.1 Docker — lifecycle	49
18.2 Docker — inspección	49

18.3 Docker — ejecución dentro de contenedores	49
18.4 Limpieza de Docker	50
18.5 Backend nativo	50
19. Apéndices	50
19.1 Resumen de puertos	50
19.2 Credenciales (entorno desarrollo)	50
19.3 Convenciones del proyecto	51
19.4 Recursos relacionados	51
19.5 Pendientes técnicos detectados	51
19.6 Cambios respecto a versiones anteriores	52

1. Introducción

Este manual unifica todo lo necesario para poner a alguien nuevo a trabajar sobre el proyecto Contrabajo: desarrollo en local con código fuente, despliegue del stack completo con Docker, y la app Android conectada a cualquiera de los dos.

1.1 Cómo está organizado este manual

El documento se divide en cinco partes lógicas. Cada parte es relativamente independiente, así que puedes saltar a la que te interese.

Parte	Capítulos	Para quién
Parte I — Bases	1, 2	Todos. Contexto y pre-requisitos.
Parte II — Ruta Nativa (desarrollo)	3 a 8	Desarrolladores activos del backend. Editar, debuggear, iterar.
Parte III — Ruta Docker (correr el stack)	9 a 12	Evaluadores, nuevos integrantes, demos. Levantar todo con pocos comandos.
Parte IV — Frontend Android	13	Quien compila la app móvil.
Parte V — Operación, troubleshooting y apéndices	14 a 19	Día a día, errores, comandos, credenciales, glosario.

1.2 Glosario rápido

Término	Significado
contrabajo-backend	Repositorio principal con el código fuente de los 4 microservicios + frontend Android.
contrabajo-deploy	Repositorio orquestador (github.com/NikoGox/contrabajo-deploy). Contiene SOLO el docker-compose.yml + .env.example. Lo que clonas para correr el stack sin compilar.
contrabajo-config	Repositorio privado (github.com/IPacal/contrabajo-config) con la configuración remota que lee el Config Server.
ruparada	Organización en Docker Hub donde están publicadas las imágenes pre-construidas (ruparada/contrabajo-config-server:1.0.0, etc.).
Config Server	Microservicio Spring Cloud Config en :8888. Provee configuración (datasources, JWT) a los 3 APIs leyéndola de contrabajo-config.
Ruta nativa	Correr los 4 microservicios desde tu IDE (VS Code) contra una BD local. Lenta para arrancar, rápida para iterar.
Ruta Docker	Correr todo el stack en contenedores con imágenes pre-construidas desde Docker Hub. Rápida para arrancar, requiere rebuild para iterar.

Término	Significado
Dev Tunnels	Funcionalidad de VS Code para exponer un puerto local a internet con URL HTTPS pública. Útil para conectar el celular físico al backend nativo.

1.3 Comparación rápida: cuándo usar cada ruta

Aspecto	Ruta Nativa	Ruta Docker
Tiempo de primer arranque	5–15 min (instalar JDK/SQL Server)	5–10 min (pull de imágenes)
Tiempo de re-arranque	10–30 s	10–30 s
Iterar cambios en código	Instantáneo (Spring DevTools)	Lento (requiere rebuild de imagen)
Breakpoints	Sí, directos	Sí, vía JDWP remote (sección 18.3)
Requiere instalar SQL Server local	Sí (o usar Docker solo para DB)	No
Buena para demos	No (depende de tu setup)	Sí (reproducible en cualquier PC)
Buena para evaluación	Si el evaluador tiene todo instalado	Sí, recomendada
Buena para desarrollo activo	Sí, recomendada	Solo para validar antes de mergear

1.4 Cambios desde la versión anterior (v4)

- **Nuevo:** ruta Docker reescrita usando imágenes pre-construidas en Docker Hub (ruparada/...:tag) y repo orquestador contrabajo-deploy.
- **Nuevo:** sección de conexión SSMS al SQL Server dockerizado.
- **Nuevo:** errores documentados con incidentes reales del equipo (env vs .env, PowerShell sin &&, JDBC localhost:1433 dentro del contenedor).
- **Nuevo:** diagnóstico estructurado para problemas de conexión Android (emulador + físico).
- **Mantenido:** ruta nativa con VS Code y Dev Tunnels, creación manual de las 3 BDs con los scripts SQL versionados.
- **Convención:** este manual (v5) es la fuente formal de referencia. Para cambios rápidos del día a día, actualizar primero el Manual Docker .md operativo; este .docx se sincroniza por hitos.

2. Pre-requisitos comunes a ambas rutas

2.1 Software

Software	Versión mínima	Para qué ruta
Git	2.40+	Ambas
Docker Desktop	4.30+ con WSL 2 (Windows)	Docker + base de datos en nativa
JDK Temurin	21 LTS	Nativa (compilar y correr el backend)
VS Code	Última estable	Nativa (editor primario)
Android Studio	Koala (2024.1)+	Frontend Android
Android SDK	API 24 mínimo, API 36 compileSdk	Frontend Android
Postman (opcional)	Última estable	Probar APIs
PowerShell 7 (opcional)	7.4+	Launcher.ps1 nativo
SSMS (opcional)	20+	Inspección/edición manual de la BD

2.2 Hardware sugerido

- RAM: 16 GB mínimo (16 GB para WSL recomendado, 8 GB es lo justo).
- CPU: 4 núcleos / 8 hilos.
- Disco: 30 GB libres (Android SDK, imágenes Docker, .gradle caches).
- Virtualización habilitada en BIOS (necesaria para WSL 2 y emulador Android).

2.3 Cuentas y accesos

- Cuenta GitHub con acceso al repositorio del proyecto.
- Acceso al repo privado `github.com/lPacal/contrabajo-config` (solicitar al líder del proyecto).
- **Personal Access Token (PAT)** de GitHub con scope `repo`. Cómo obtenerlo: GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token. Copiarlo inmediatamente (no se vuelve a mostrar).

PARTE II — Ruta Nativa (desarrollo)

Esta es la ruta recomendada para el día a día de desarrollo: editar código, debuggear con breakpoints, recompilar al guardar, y exponer puertos del backend a dispositivos físicos vía Dev Tunnels.

3. Clonar el proyecto

Clonar el repositorio principal con el código fuente:

```
git clone <URL-del-repositorio> contrabajo
cd contrabajo
ls
# Contrabajo Backend/    ← 4 microservicios Spring Boot
# Contrabajo/            ← proyecto Android
```

4. Instalar y configurar VS Code

VS Code es el editor primario del backend. Permite exponer puertos vía Dev Tunnels (sección 7) para que dispositivos físicos o el resto del equipo puedan probar el backend en tu máquina sin tocar firewall/router.

4.1 Instalación

1. Descargar e instalar VS Code desde <https://code.visualstudio.com/>
2. Verificar que el JDK 21 esté instalado: `java -version`

4.2 Extensiones obligatorias

Extensión	ID	Para qué
Extension Pack for Java	vscjava.vscode-java-pack	Lenguaje, debug, Maven, gestor de proyectos.
Spring Boot Extension Pack	vmware.vscode-boot-dev-pack	Spring Boot Dashboard, Initializr, Tools.
Lombok Annotations Support	gabrielbb.vscode-lombok	Sin esto las anotaciones <code>@Data</code> , <code>@Slf4j</code> salen como errores.
DotENV	mikestead.dotenv	Colorea archivos <code>.env</code> .
SonarLint (opcional)	sonarsource.sonarlint-vscode	Detecta code smells y bugs en tiempo real.
GitLens (opcional)	eamodio.gitlens	Blame inline, historial enriquecido.

Instalación rápida de todas por terminal:

```
code --install-extension vscjava.vscode-java-pack
code --install-extension vmware.vscode-boot-dev-pack
code --install-extension gabrielbb.vscode-lombok
code --install-extension mikestead.dotenv
```

4.3 Abrir el proyecto en VS Code

Abrir la carpeta completa del backend, no cada microservicio por separado:

```
cd "Contrabajo Backend"  
code .
```

La primera vez VS Code descarga el Java Language Server (~3 min). Cuando termine, en la barra de estado aparece 'Java: Ready'.

Si VS Code no detecta el JDK 21

Ctrl+, → buscar java.jdt.ls.java.home → poner la ruta del JDK (ej. C:\Program Files\Eclipse Adoptium\jdk-21).

Alternativa: agregar al settings.json del workspace java.configuration.runtimes con name JavaSE-21, path al JDK, default true.

Recargar: F1 → 'Java: Clean Java Language Server Workspace'.

5. Crear las bases de datos

Hibernate corre con ddl-auto: validate, así que las tres bases (MS_Usuarios, MS_Servicios, MS_Comunicaciones) y todas sus tablas deben existir ANTES de levantar los servicios. Los scripts SQL están versionados en el repo de documentación.

5.1 Ubicación de los scripts

Dentro del proyecto: Contrabajo v0/contrabajo-documentation/Producto/database/. Estructura:

```

Producto/database/
├── ms-usuarios/
│   ├── 01-ms-usuarios.sql ←
│   └── crea DB + tablas
│       ├── queries/seed_comunas_rm.sql ←
│       └── comunas de la RM
│           ├── sp/ ← 7
│           └── stored procedures
│               ├── ms-servicios/
│               │   ├── 02-ms-servicios.sql ← 5
│               │   └── sp/
│               └── SPS
│                   ├── ms-comunicaciones/
│                   │   ├── 03-ms-comunicaciones.sql ← 8
│                   └── sp/
└── SPS

```

Los scripts principales son destructivos

01-, 02- y 03- hacen DROP DATABASE si la base ya existe.

Está bien en desarrollo (parten limpias en cada corrida). NUNCA ejecutar sobre datos que quieras conservar.

5.2 Pre-requisitos

3. SQL Server corriendo (instancia local Developer en :58166 o contenedor Docker en :1433).
4. Cliente SQL conectado: SSMS, Azure Data Studio, DBeaver o sqlcmd.
5. Credenciales con permiso para crear bases: usuario 'sa' o sysadmin.

5.3 Crear el login admincontrabajo

Ejecutar UNA SOLA VEZ en el servidor, antes de los scripts principales:

```

USE master;
GO
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'admincontrabajo')
BEGIN
    CREATE LOGIN admincontrabajo WITH PASSWORD = 'Contrabajo123!', CHECK_POLICY = OFF;
END
GO

```

5.4 Orden de ejecución

Paso	Script	Qué hace
1	ms-usuarios/01-ms-usuarios.sql	Crea MS_Usuarios + tablas.
2	ms-usuarios/queries/seed_comunas_rm.sql	Inserta comunas de la Región Metropolitana.
3	ms-servicios/02-ms-servicios.sql	Crea MS_Servicios + tablas.
4	ms-comunicaciones/03-ms-comunicaciones.sql	Crea MS_Comunicaciones + tablas.
5	ms-usuarios/sp/*.sql (7 archivos)	Stored procedures usuarios.
6	ms-servicios/sp/*.sql (5)	Stored procedures servicios.
7	ms-comunicaciones/sp/*.sql (8)	Stored procedures comunicaciones.
8	(snippet de permisos abajo)	Vincula admincontrabajo a las 3 bases como db_owner.

5.5 Otorgar permisos al login

```
USE MS_Usuarios;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'admincontrabajo')
  CREATE USER admincontrabajo FOR LOGIN admincontrabajo;
EXEC sp_addrolemember 'db_owner', 'admincontrabajo';
GO

USE MS_Servicios;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'admincontrabajo')
  CREATE USER admincontrabajo FOR LOGIN admincontrabajo;
EXEC sp_addrolemember 'db_owner', 'admincontrabajo';
GO

USE MS_Comunicaciones;
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'admincontrabajo')
  CREATE USER admincontrabajo FOR LOGIN admincontrabajo;
EXEC sp_addrolemember 'db_owner', 'admincontrabajo';
GO
```

5.6 Ejecución con sqlcmd (PowerShell)

```
cd "Contrabajo v0/contrabajo-documentation/Producto/database"

sqlcmd -S localhost,1433 -U sa -P $env:SA_PASSWORD -C -i "ms-usuarios/01-ms-usuarios.sql"
sqlcmd -S localhost,1433 -U sa -P $env:SA_PASSWORD -C -i
"ms-usuarios/queries/seed_comunas_rm.sql"
sqlcmd -S localhost,1433 -U sa -P $env:SA_PASSWORD -C -i "ms-servicios/02-ms-servicios.sql"
sqlcmd -S localhost,1433 -U sa -P $env:SA_PASSWORD -C -i
"ms-comunicaciones/03-ms-comunicaciones.sql"

# Stored procedures en bloque
```

```
Get-ChildItem ms-usuarios/sp/*.sql | ForEach-Object { sqlcmd -S localhost,1433 -U sa
-P $env:SA_PASSWORD -C -i $_.FullName }
Get-ChildItem ms-servicios/sp/*.sql | ForEach-Object { sqlcmd -S localhost,1433 -U sa
-P $env:SA_PASSWORD -C -i $_.FullName }
Get-ChildItem ms-comunicaciones/sp/*.sql | ForEach-Object { sqlcmd -S localhost,1433 -U sa
-P $env:SA_PASSWORD -C -i $_.FullName }
```

5.7 Verificación

```
-- 1. Las 3 bases existen
SELECT name FROM sys.databases WHERE name LIKE 'MS_%';

-- 2. Tablas
USE MS_Usuarios;      SELECT name FROM sys.tables ORDER BY name;
USE MS_Servicios;     SELECT name FROM sys.tables ORDER BY name;
USE MS_Comunicaciones; SELECT name FROM sys.tables ORDER BY name;

-- 3. Seed de comunas (esperado: > 30)
USE MS_Usuarios; SELECT COUNT(*) FROM dbo.comuna;

-- 4. Stored procedures (esperado: 7 + 5 + 8 = 20)
SELECT COUNT(*) AS total_sp FROM (
  SELECT 1 AS c FROM MS_Usuarios.sys.procedures
  UNION ALL SELECT 1 FROM MS_Servicios.sys.procedures
  UNION ALL SELECT 1 FROM MS_Comunicaciones.sys.procedures
) t;
```

6. Configurar variables de entorno (.env + launch.json)

Crear un .env en la raíz del backend con las credenciales para el Config Server y un .vscode/launch.json con una configuración por servicio.

6.1 Archivo .env

```
# --- Config Server (lee de
github.com/lPacal/contrabajo-config)
---
GIT_USERNAME=<tu-usuario-github>
GIT_PASSWORD=<tu-personal-access-token
>

# --- SQL Server local (instancia
Developer en :58166 o Docker en :1433)
---
DB_HOST=localhost
DB_PORT=58166      # o 1433 si usas
SQL Server en Docker
DB_USER=admincontrabajo
DB_PASSWORD=Contrabajo123!

# --- JWT ---
JWT_SECRET=EstaEsUnaClaveSecretaMuyLar
gaParaQueNadieLaAdivine123456789
```

```
# --- Coordinación entre servicios
(nativo: localhost) ---
CONFIG_SERVER_URL=http://localhost:8888
```

Buenas prácticas con el .env

NUNCA versionarlo. Agregar .env al .gitignore.
 Mantener un .env.example versionado con las llaves pero sin valores.
 Cada desarrollador tiene su propio .env con su propio PAT.

6.2 .vscode/launch.json

Una configuración por servicio + un compound para arrancar los 4 con un solo F5:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "java",
      "name": "Config Server",
      "request": "launch",
      "mainClass":
        "com.contrabajo.config_server.ConfigServerApplication",
      "projectName": "config-server",
      "envFile": "${workspaceFolder}/.env"
    },
    {
      "type": "java",
      "name": "Usuarios API",
      "request": "launch",
      "mainClass":
        "com.contrabajo.usuarios_api.UsuariosApiApplication",
      "projectName": "usuarios_api",
      "envFile": "${workspaceFolder}/.env"
    }
    // ... idem servicios_api y comunicaciones_api
  ],
  "compounds": [
    {
      "name": "Backend completo",
      "configurations": [
        "Config Server",
        "Usuarios API",
        "Servicios API",
        "Comunicaciones API"
      ]
    }
  ]
}
```

Sobre el compound 'Backend completo'

VS Code lanza los 4 casi en paralelo. Si Config Server no alcanza a estar listo antes que los APIs intenten leer su config, fallarán con fail-fast.
 En la práctica: arranca Config Server primero (F5 sobre 'Config Server'), espera 'Started'

ConfigServerApplication', y recién lanza los otros tres (o el compound).
 Alternativa: Spring Boot Dashboard (sección 7.1) que respeta mejor el orden.

7. Arrancar los servicios nativos

Cuatro opciones, de más a menos cómoda:

7.1 Spring Boot Dashboard (VS Code, recomendado)

6. Abrir el sidebar de Spring Boot Dashboard (ícono de hoja verde).
7. Click en ► junto a 'config-server'.
8. Esperar 'Started ConfigServerApplication in X seconds'.
9. Click en ► junto a los otros tres (en paralelo). Para debug: ícono .

7.2 Run and Debug (F5)

Ctrl+Shift+D → elegir configuración del dropdown → F5.

7.3 Contrabajo Launcher.ps1 (Windows)

Script PowerShell con menú interactivo, healthchecks de puertos y stop seguro. Desde Contrabajo Backend/:

```
powershell -ExecutionPolicy Bypass -File ".\Contrabajo Launcher.ps1"
```

Arranca en orden Config Server → Usuarios → Servicios → Comunicaciones, esperando que cada uno levante antes del siguiente.

7.4 Maven desde terminal

```
# Terminal 1
cd contrabajo-config-server && ./mvnw spring-boot:run

# Terminal 2 (cuando config-server esté arriba)
cd contrabajo-usuarios-api && ./mvnw spring-boot:run
# Idem para servicios-api y comunicaciones-api
```

8. Exponer puertos con Dev Tunnels

Permite que dispositivos físicos o compañeros prueben tu backend nativo sin tocar firewall ni router.

8.1 Crear un túnel

10. Con el servicio corriendo localmente, abrir el panel 'Ports' (Ctrl+`, pestaña Ports).
11. Click en 'Forward a Port' → ingresar 8081 (o el que necesites).
12. Autenticarse con GitHub/Microsoft la primera vez.
13. Aparece URL <https://xxxx-8081.use.devtnnls.ms> — esa es la URL pública.

14. Para que otros la usen: clic derecho → Port Visibility → Public.
15. Detener: clic derecho → Stop Forwarding, o cerrar VS Code.

8.2 Exponer los 4 puertos a la vez

Repetir con 8081, 8082, 8083 y 8888 si necesitas Config Server público.

Importante con Dev Tunnels

Las URLs públicas son accesibles para cualquiera que las tenga. No dejar el túnel abierto cuando no se necesita.

URLs son temporales: cambian al cerrar VS Code.

URL persistente: instalar CLI devtunnel (winget install Microsoft.devtunnel) y crear túnel con nombre fijo.

Latencia: +100–300 ms vs LAN directa por el hop de Microsoft.

8.3 Usar la URL en el frontend Android

Editar app/build.gradle.kts:

```
buildConfigField("String", "USUARIOS_BASE_URL",  
    "\"https://xxxx-8081.use.devtunnels.ms/\"")  
buildConfigField("String", "SERVICIOS_BASE_URL",  
    "\"https://yyyy-8082.use.devtunnels.ms/\"")  
buildConfigField("String", "COMUNICACIONES_BASE_URL",  
    "\"https://zzzz-8083.use.devtunnels.ms/\"")
```

PARTE III — Ruta Docker (correr el stack)

Ruta recomendada para evaluadores, nuevos integrantes y demos. Usa imágenes pre-construidas en Docker Hub y un repositorio orquestador separado del código fuente.

9. Configurar memoria WSL para Docker

Paso obligatorio antes del primer arranque

Docker Desktop en Windows usa WSL 2 como backend. WSL asigna por defecto ~3 GB de RAM, lo que no alcanza para SQL Server + 4 servicios Java.

Si te saltas este paso: 'sqlservr: This program requires a machine with at least 2000 megabytes of memory.'

9.1 Crear o actualizar .wslconfig

```
@[wsl2]
memory=16GB
processors=4
"@ | Out-File -FilePath "$env:USERPROFILE\.wslconfig" -Encoding utf8
```

Ajusta memory a un cuarto de tu RAM total (16 GB si tienes 48 GB; 4 GB es el mínimo absoluto).

9.2 Verificar

```
Get-Content "$env:USERPROFILE\.wslconfig"
```

9.3 Aplicar

16. Buscar la ballena 🐳 en la bandeja del sistema → clic derecho → Quit Docker Desktop.

17. Si no cierra con el botón normal, matarlo por la fuerza:

```
Stop-Process -Name "Docker Desktop" -Force
Stop-Process -Name "com.docker.backend" -Force
Stop-Process -Name "dockerd" -Force
```

18. Apagar WSL:

```
wsl --shutdown
```

19. Reabrir Docker Desktop y esperar a que la ballena vuelva en estado 'running'.

10. Clonar el repo orquestador y configurar credenciales

La ruta Docker NO usa el repo del código fuente, usa un repositorio separado (contrabajo-deploy) que solo contiene el docker-compose.yml y .env.example. El compose hace pull de las imágenes ya construidas desde Docker Hub.

10.1 Clonar el repo orquestador

```
git clone https://github.com/NikoGox/contrabajo-deploy.git
cd contrabajo-deploy

# Verificar contenido
ls
# .env.example
# docker-compose.yml
# (a veces docker/ con scripts SQL)
```

10.2 Crear el .env

```
# En PowerShell o cmd dentro de contrabajo-deploy/
copy .env.example .env
```

ATENCIÓN: el archivo debe llamarse exactamente .env

Casos reales reportados por el equipo:

- env (sin punto) → Docker Compose no lo lee.
- .env.txt → invisible para Compose, Windows oculta la extensión.

Verificar con: dir .env (PowerShell) o ls -la .env (Git Bash). Debe aparecer el archivo con punto inicial.

10.3 Obtener Personal Access Token de GitHub

El Config Server clona contrabajo-config (privado). GitHub no acepta contraseña — necesitas un PAT.

20. github.com → foto perfil → Settings.
21. Developer settings (al final del menú izquierdo).
22. Personal access tokens → Tokens (classic) → Generate new token (classic).
23. Nombre: contrabajo-config-server
24. Expiration: lo que necesites (o 'No expiration' para desarrollo).
25. Scopes: marcar **repo** (incluye lectura de privados).
26. Generate token → **COPIAR INMEDIATAMENTE** (no se vuelve a mostrar). Guardarlo en un gestor de contraseñas.

10.4 Completar el .env

Editar .env con los valores reales. Las 4 variables son obligatorias:

```
# Docker Hub — qué imágenes
pull
DOCKER_ORG=ruparada
DOCKER_TAG=1.0.0

# GitHub — para que Config
Server clone
contrabajo-config
GIT_USERNAME=lPacal
```

GIT_PASSWORD=ghp_XXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXXXXXXXXXX	
Variable	Para qué
DOCKER_ORG	Organización en Docker Hub donde están las imágenes. Por convención: ruparada.
DOCKER_TAG	Versión específica a pullear (ej. 1.0.0). El compose interpola ruparada/contrabajo-config-server:1.0.0, etc.
GIT_USERNAME	Tu usuario de GitHub.
GIT_PASSWORD	Tu Personal Access Token (NO tu contraseña de GitHub).

11. Levantar el stack con Docker

11.1 Primer arranque

Desde la carpeta contrabajo-deploy con el .env listo:

```
docker compose pull      #
descarga las 4 imágenes desde
Docker Hub
docker compose up -d     # las
levanta en segundo plano
docker compose ps        #
verifica estado
```

Si tu PowerShell no soporta &&

Algunas versiones antiguas de Windows PowerShell rechazan && con: 'El token & no es un separador de instrucciones válido'.

Solución: usar ; o líneas separadas:

docker compose pull; docker compose up -d; docker compose ps
(PowerShell 7+ sí soporta && y es lo recomendado para nuevos setups.)

La primera vez tarda 5–10 minutos por la descarga de imágenes (~3 GB entre SQL Server y los Java).

11.2 Veces siguientes

```
docker compose up -d      # levanta sin re-pull
docker compose logs -f    # ver logs en vivo
```

11.3 Actualizar a una versión nueva

Cuando salga una nueva versión de imágenes (DOCKER_TAG=1.0.1, por ejemplo):

27. Editar .env y cambiar DOCKER_TAG.

28. Ejecutar: docker compose pull && docker compose up -d

29. Verificar: docker compose ps

11.4 Levantar desde Docker Desktop (sin terminal)

30. Abrir Docker Desktop → Settings → Resources → File sharing → agregar la carpeta contrabajo-deploy si pide permisos.
31. Volver a la pestaña 'Containers'.
32. Buscar el proyecto 'contrabajo-deploy' o usar `docker compose up -d` desde una terminal dentro de la carpeta.
33. Para revisar salida de cada servicio: clic sobre el contenedor → pestaña Logs.

11.5 Orden de arranque automático

```
sqlserver (healthcheck SELECT 1)
└─▶ sqlserver-init (crea login, BDs, tablas, índices) → Exited (0)
    └─▶ config-server (healthcheck /actuator/health)
        └─▶ usuarios-api (8081)
            └─▶ servicios-api (8082)
                └─▶ comunicaciones-api (8083)
```

12. Verificar y operar el stack Docker

12.1 Desde Docker Desktop

```
▶ contrabajo-sqlserver          Running    1433/tcp
✓ contrabajo-sqlserver-init      Exited (0) ← normal
▶ contrabajo-config-server      Running    8888/tcp
▶ contrabajo-usuarios-api       Running    8081/tcp
▶ contrabajo-servicios-api      Running    8082/tcp
▶ contrabajo-comunicaciones-api  Running    8083/tcp
```

sqlserver-init con Exited (0) es comportamiento esperado: corrió init.sql una vez y terminó.

12.2 Endpoints y Swagger

Servicio	Health check	Swagger UI
config-server	<code>http://localhost:8888/actuator/health</code>	—
usuarios-api	<code>http://localhost:8081/actuator/health</code>	<code>http://localhost:8081/swagger-ui/index.html</code>
servicios-api	<code>http://localhost:8082/actuator/health</code>	<code>http://localhost:8082/swagger-ui/index.html</code>
comunicaciones-api	<code>http://localhost:8083/actuator/health</code>	<code>http://localhost:8083/swagger-ui/index.html</code>

12.3 Verificar las 3 BDs

```
docker exec -it contrabajo-sqlserver \
/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P 'Contrabajo123!' -No \
-Q "SELECT name FROM sys.databases WHERE name LIKE 'MS_%';"
```

12.4 Detener el stack

```
docker compose down          # conserva datos en volumen sqlserver-data
docker compose down -v       # borra volumen → próximo up re-ejecuta init.sql
docker compose down --rmi local # + borra imágenes locales (forzar re-pull)
```

12.5 Persistencia de datos

SQL Server guarda en el volumen sqlserver-data. Sobrevive a down sin -v, se borra con -v.

PARTE IV — Frontend Android

El proyecto Android no se dockeriza. Se compila e instala desde Android Studio.

13. Configurar y compilar la app Android

13.1 Abrir el proyecto

34. Abrir Android Studio.
35. File → Open → seleccionar la carpeta Contrabajo/ (dentro de Contrabajo v0/).
36. Esperar a que Gradle sincronice (~5 min la primera vez).
37. Verificar local.properties: debe apuntar al Android SDK.

13.2 Configurar URLs según escenario

La app define URLs base por buildConfigField en app/build.gradle.kts:

Escenario	URL base	Detalle
Emulador + backend Docker en tu PC	http://10.0.2.2:8081/	10.0.2.2 es el alias del host desde el emulador (default actual).
Emulador + backend nativo en tu PC	http://10.0.2.2:8081/	Igual que el caso anterior.
Físico LAN + backend Docker en PC	http://192.168.X.Y:8081/	Reemplazar X.Y por la IPv4 del PC. Verificar con ipconfig (Windows) → IPv4 del adaptador WiFi/Ethernet.
Físico vía Dev Tunnels	https://xxxx-8081.use.dev.tunnels.ms/	URL pública generada por VS Code (sección 8). No requiere LAN compartida ni abrir firewall.
Físico vía servidor producción	https://api.contrabajo.cl/	(Cuando exista). HTTPS sin cleartext.
Importante con celular físico vía LAN NO usar 'localhost' en el celular (apunta al propio celular). NO usar IPs de adaptadores VPN/ZeroTier/Radmin para pruebas LAN normales — usar la IPv4 del WiFi/Ethernet directo. Firewall de Windows debe permitir entradas en 8081/8082/8083 (y 8888 si necesitas Config Server público). Si la URL es HTTP plano: AndroidManifest.xml necesita		

Escenario	URL base	Detalle
	android:usesCleartextTraffic="true". Verificar conexión desde el navegador del celular: <code>http://<IP-LAN>:8081/swagger-ui/index.html</code> — si no abre ahí, no es problema de Retrofit ni del código Android, es red/firewall.	

13.3 Compilar y ejecutar

38. Crear un AVD: Tools → Device Manager → Create Virtual Device → Pixel 6 / API 34+.
39. Run 'app' (Shift+F10).
40. Para APK firmado: Build → Generate Signed Bundle / APK → seguir wizard.

PARTE V — Operación, troubleshooting y apéndices

14. Conectar SSMS a SQL Server (Docker)

SQL Server Management Studio se usa para inspeccionar/editar la BD manualmente. Para conectarse al contenedor:

Campo	Valor
Server type	Database Engine
Server name	localhost,1433 (o solo localhost)
Authentication	SQL Server Authentication
Login	sa
Password	Contrabajo123!
Encrypt	Optional
Trust server certificate	Activado ✓
	Importante NO usar 'NOMBREPC\SQLEXPRESS' para conectarse al SQL Server de Docker — ese nombre apunta a tu instancia local nativa, no al contenedor. Si 'sa' falla, probar con el usuario de aplicación: admincontrabajo / Contrabajo123!

15. Verificación end-to-end

Checklist para confirmar que todas las piezas conversan. Si alguna falla, ir a la sección 17.

Paso	Acción	Esperado
1	docker compose ps (o Spring Boot Dashboard en VS Code)	Todos los contenedores Running (o los 4 procesos Java en Java Ready).
2	curl http://localhost:8888/actor/health	{"status":"UP"}
3	curl http://localhost:8081/actor/health (8082, 8083)	{"status":"UP"} en los tres.
4	En SSMS: SELECT name FROM sys.databases	MS_Usuarios, MS_Servicios, MS_Comunicaciones.
5	Postman: importar Contrabajo-Backend.postman_collection.json → Auth → Register	201 Created.

Paso	Acción	Esperado
6	Postman: Auth → Login	200 OK con token JWT.
7	App en emulador: registrar y entrar	Navegación al home, sin errores.

16. Flujo de desarrollo con Docker

Si decides desarrollar contra el stack Docker (no contra los procesos nativos), estos patrones ayudan.

16.1 Cuándo recompilar/repullar

Cambio	Acción
Salió nueva versión de imágenes (DOCKER_TAG)	Actualizar .env, docker compose pull, docker compose up -d
Editaste docker-compose.yml	docker compose up -d (sin pull; recrea contenedores)
Editaste init.sql (si tienes acceso al de deploy)	docker compose down -v && docker compose up -d
Editaste .env (credenciales Git)	docker compose restart config-server
Cambios solo en código Android	No tocar Docker. Recompilar en Android Studio.
Estás desarrollando el código backend	Usar ruta nativa (Parte II), Docker es solo para correr lo ya publicado.

16.2 Acceder a una consola dentro de un contenedor

```
# Bash en SQL Server
docker exec -it contrabajo-sqlserver bash

# sqlcmd directo
docker exec -it contrabajo-sqlserver \
  /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P 'Contrabajo123!' -No

# Bash en un microservicio
docker exec -it contrabajo-usuarios-api bash

# Variables de entorno del contenedor
docker exec contrabajo-config-server env | grep GIT_
```

16.3 Debug remoto JVM desde VS Code (sobre contenedores)

Solo aplica si construyes tu propia imagen (no las publicadas). Las imágenes ruparada/...:tag oficiales no tienen JDWP.

Para breakpoints en contenedores propios: agregar al ENTRYPOINT -agentlib:jdwp..., exponer puerto 5005 en compose, y attach en launch.json. Detalles completos en sección 18.3.

16.4 Logs filtrados

```

docker compose logs -f                                # todos
docker compose logs -f --tail=100 usuarios-api         # uno, últimas 100 + vivo
docker compose logs usuarios-api | grep -i "error\|exception"
docker compose logs --timestamps -f                   # con timestamps

```

17. Errores comunes y soluciones

17.1 Memoria / WSL / Docker Desktop

Error	Causa	Solución
This program requires at least 2000 megabytes of memory	WSL con poca RAM.	Sección 9: editar .wslconfig con memory=16GB, wsl --shutdown, reabrir Docker.
Docker Desktop no cierra con el botón normal	Proceso backend se reinicia solo.	Stop-Process en Docker Desktop / com.docker.backend / dockerd; luego wsl --shutdown.
.wslconfig no se aplica	Archivo mal ubicado o WSL no reiniciado.	Get-Content "\$env:USERPROFILE\.wslconfig" para confirmar. Recrear si falta. wsl --shutdown.

17.2 Variables de entorno y .env

Error	Causa	Solución
The DOCKER_ORG variable is not set / unable to get image '/contrabajo-config-server:'	Falta .env o tiene nombre incorrecto (caso real: archivo se llamaba 'env' sin punto).	Verificar con ls -la .env. Crear desde .env.example. Completar las 4 variables (DOCKER_ORG, DOCKER_TAG, GIT_USERNAME, GIT_PASSWORD).
Could not locate PropertySource / Config Server no arranca	Causa 1: .env vacío. Causa 2: PAT expirado o sin scope 'repo'.	Verificar .env. Si PAT vencido, regenerar en GitHub con scope repo y actualizar .env.
docker compose pull && docker compose up -d falla con 'token && no es separador válido'	PowerShell viejo no soporta &&.	Usar ; o líneas separadas: docker compose pull; docker compose up -d; docker compose ps. Mejor instalar PowerShell 7+.
docker pull manual funciona pero compose no	docker pull baja imágenes pero compose no resuelve variables si .env está mal.	Corregir .env. Volver a ejecutar docker compose up -d.

17.3 BDs y SQL Server

Error	Causa	Solución
Login failed for user 'admincontrabajo'	Los APIs intentaron conectar antes de que sqlserver-init creara el login.	El compose ya tiene depends_on: sqlserver-init: condition: service_completed_successfully. Si persiste: docker compose down -v && docker compose up -d.
APIs en loop con TCP/IP connection to localhost:1433 has failed (dentro del contenedor)	'localhost' dentro de un contenedor apunta al propio contenedor, no a SQL Server. Una propiedad remota (Config Server) puede estar pisando la URL.	Forzar SPRING_DATASOURCE_URL en docker-compose.yml apuntando a sqlserver:1433. Ver ejemplo abajo.
sqlserver-init Exited con código != 0	init.sql falló (estado incompatible).	docker compose logs sqlserver-init. docker compose down -v && docker compose up -d.
Schema-validation: missing table [dbo.usuario] (ruta nativa)	Las BDs no tienen esquema (ddl-auto=validate).	Correr los scripts SQL de la sección 5.
Restaurar .bak falla: 'Database backed up on more recent version'	El .bak es de un SQL Server más nuevo que el del contenedor.	Usar Export Data en SSMS, BCP, o regenerar con scripts.

Snippet para forzar datasource explícito en el compose (ruta Docker, mitigación del JDBC localhost):

```

usuarios-api:
  environment:
    SPRING_DATASOURCE_URL:
jdbc:sqlserver://sqlserver:1433;databaseName=MS_Usuarios;encrypt=true;trustServerCertificate=true;loginTimeout=30;
    SPRING_DATASOURCE_USERNAME: admincontrabajo
    SPRING_DATASOURCE_PASSWORD: Contrabajo123!
# Repetir cambiando databaseName para servicios-api y comunicaciones-api

```

17.4 Conexión Android / red

Diagnóstico estructurado cuando la app móvil no conecta y Docker 'parece bien':

41. Confirmar backend arriba: docker compose ps muestra contenedores en Up.
42. Swagger en host responde 200 en localhost:8081/8082/8083.
43. Emulador: usar 10.0.2.2 (no IP LAN, no localhost).
44. Físico LAN: usar IP_LAN_PC (ej. 192.168.100.66). NO IPs de VPN/ZeroTier.
45. AndroidManifest.xml: android:usesCleartextTraffic="true" + permiso INTERNET.
46. Firewall de Windows permite puertos 8081/8082/8083/8888.
47. Celular y PC en la misma red/subred.
48. Prueba mínima desde el navegador del celular:
http://IP_LAN_PC:8081/swagger-ui/index.html. Si no abre ahí, NO es problema de código Android, es red/firewall.

17.5 Otros

Error	Causa	Solución
Connection refused al abrir Swagger	Contenedor todavía inicializando.	Esperar 60–90 s tras Running.
bind: address already in use	Algo usa 1433/8081/8082/8083/8888 localmente.	docker compose down. Detener SQL Server local. Verificar con netstat.
VS Code 'Java: Ready' pero Spring Boot Dashboard vacío	Workspace no detectó los pom.xml.	F1 → Java: Clean Java Language Server Workspace → Restart and delete. Luego F1 → Spring Boot Dashboard: Refresh.
Build Android falla: SDK location not found	local.properties con ruta inválida.	Reabrir Android Studio o editar local.properties.
INSTALL_FAILED_IN_SUFFICIENT_STORAGE en emulador	AVD sin espacio.	Wipe data del AVD o crear uno nuevo más grande.

18. Referencia rápida de comandos

18.1 Docker — lifecycle

```

docker compose pull                # bajar imágenes (ruta deploy)
docker compose up -d               # detached
docker compose up -d --build       # con build (si tienes Dockerfiles)
docker compose up -d --build usuarios-api # solo un servicio

docker compose down               # detener, conservar datos
docker compose down -v            # detener + borrar volumen
docker compose down --rmi local   # + borrar imágenes locales

docker compose restart config-server

```

18.2 Docker — inspección

```

docker compose ps
docker compose logs -f
docker compose logs -f --tail=100 usuarios-api
docker stats

```

18.3 Docker — ejecución dentro de contenedores

```

docker exec -it contrabajo-sqlserver bash
docker exec -it contrabajo-sqlserver \
  /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P 'Contrabajo123!' -No
docker exec -it contrabajo-usuarios-api bash
docker exec contrabajo-config-server env | grep GIT_

```

18.4 Limpieza de Docker

```
docker system df          # uso de espacio
docker system prune      # parados, redes, caches
docker system prune -a --volumes  # + huérfanos (cuidado)
docker volume rm contrabajo-deploy_sqlserver-data
```

18.5 Backend nativo

```
# Maven directo
cd contrabajo-config-server && ./mvnw spring-boot:run

# PowerShell Launcher
powershell -ExecutionPolicy Bypass -File ".\Contrabajo Launcher.ps1"

# Verificar puertos ocupados (Windows)
netstat -ano | findstr "1433 8081 8082 8083 8888"
```

19. Apéndices

19.1 Resumen de puertos

Puerto	Servicio	Origen
8888	Spring Cloud Config Server	Backend
8081	Usuarios API	Backend
8082	Servicios API (HTTP + WebSocket STOMP)	Backend
8083	Comunicaciones API	Backend
1433	SQL Server (Docker)	Base de datos
58166	SQL Server (instancia local Developer)	Base de datos (nativa)
5005	JDWP remote debug (opcional)	Dev

19.2 Credenciales (entorno desarrollo)

Parámetro	Valor
SA Password (Docker)	Contrabajo123!
Login de aplicación	admincontrabajo / Contrabajo123!
Host SQL (Docker, desde host)	localhost:1433
Host SQL (Docker, entre contenedores)	sqlserver:1433

Parámetro	Valor
Host SQL (nativa local)	localhost:58166
Bases	MS_Usuarios, MS_Servicios, MS_Comunicaciones
DOCKER_ORG	ruparada
DOCKER_TAG actual	1.0.0
	Solo desarrollo local No usar estas credenciales en producción ni subirlas a repos públicos. Para producción: gestor de secretos (Azure Key Vault, AWS Secrets Manager, Vault).

19.3 Convenciones del proyecto

- Branch: feature/<nombre-corto> a partir de main. PRs revisados antes de mergear.
- Commits: prefijo por módulo (usuarios:, servicios:, comunicaciones:, config:, android:, deploy:).
- Idioma del código: entidades de dominio en español (Usuario, OfertaServicio, ChatOferta).
- Editor: VS Code como primario; IntelliJ permitido como alternativa.
- Versionar .vscode/launch.json y .vscode/settings.json. NO versionar .env (solo .env.example).
- Imágenes Docker se publican con tags semánticos (1.0.0, 1.0.1, 1.1.0). DOCKER_TAG=latest está prohibido para entregas.

19.4 Recursos relacionados

- Repo principal (código): clonado en sección 3.
- Repo orquestador Docker: github.com/NikoGox/contrabajo-deploy.
- Repo de configuración: github.com/IPacal/contrabajo-config (privado).
- Docker Hub: hub.docker.com/u/ruparada (imágenes pre-construidas).
- Manual Docker .md operativo: 01_Documentacion/Especificaciones/Manual Docker - Proyecto Contrabajo.md (fuente viva para errores nuevos).
- Postman Collection: [contrabajo-backend/postman/Contrabajo-Backend.postman_collection.json](#).
- Scripts SQL versionados: [contrabajo-documentation/Producto/database/](#).
- Bitácoras de integración: 05_Codex_IA/contexto codex/.
- Diagramas: 01_Documentacion/Diagramas/.

19.5 Pendientes técnicos detectados

Lista viva de deuda técnica para cerrar en próximas iteraciones:

- Mover SA_PASSWORD, password de admincontrabajo y secret JWT a .env (hoy hardcoded en docker-compose.yml y init.sql).
- Convertir init.sql en plantilla y aplicar envsubst en init.sh para que la password de admincontrabajo venga del .env.
- Publicar README del repo contrabajo-config con qué archivos espera por servicio.

- Pipeline CI que publique automáticamente nuevas versiones de imágenes a Docker Hub cuando se taggea release en GitHub.
- Documentar cómo conectar Postman al backend dockerizado con el environment file que vive en el repo.
- Healthchecks individuales para usuarios-api / servicios-api / comunicaciones-api en el compose (hoy solo el config-server tiene).

19.6 Cambios respecto a versiones anteriores

Manual previo	Estado	Reemplazado por
Manual de Configuración v4 - con BD.docx	Legacy / referencia histórica	Este v5 (Parte II + sección 5 BD)
Manual Docker v3 - sincronizado con MD 1.1.docx	Legacy	Este v5 (Parte III + sección 17.2)
Manual Docker - Proyecto Contrabajo.md	ACTIVO — fuente viva	Sigue siendo la fuente operativa diaria. Este v5 se sincroniza desde el .md en hitos.